



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 7: Manejo Centralizado de Excepciones, Autenticación RBAC y Seguridad de Sesiones con JWT

Resumen

Esta práctica guiada complementa de forma directa los contenidos teóricos de los temas **3.4 (Manejo de Errores)**, **3.5 (Autenticación y Autorización)** y **3.6 (Seguridad en el Manejo de Sesiones)**. Usted construirá una capa de seguridad empresarial completa para una API RESTful en Java 21 utilizando Spring Boot 3.x, Spring Security 6 y JSON Web Tokens (JWT).

A lo largo del laboratorio, usted configurará un middleware centralizado para la captura de excepciones con `@RestControllerAdvice` respondiendo bajo la especificación RFC 7807 (Problem Details), mapeará usuarios y roles relacionales en SQL Server con encriptación BCrypt, estructurará la cadena de filtros `SecurityFilterChain`, e implementará un proveedor y filtro personalizado de JWT para gestionar sesiones totalmente sin estado (*Stateless*).

Metadatos del Laboratorio

- **Tiempo estimado:** 4 horas.
- **Herramientas requeridas:** Java 21 (o versión local instalada), Spring Boot 3.x, Microsoft SQL Server Developer Edition, SSMS, Maven, VS Code, Postman o cURL.

Metas de Aprendizaje:

1. Implementar un manejador global de excepciones con `@RestControllerAdvice` y DTOs estandarizados para capturar errores de validación y de negocio sin revelar stack traces del servidor.
2. Configurar Spring Security 6 con `BCryptPasswordEncoder`, `UserDetailsService` y control de acceso basado en roles (RBAC) declarativo y por anotaciones (`@PreAuthorize`).
3. Construir un proveedor de tokens JWT (`JwtTokenProvider`) y un filtro de interceptación (`JwtAuthenticationFilter`) para autenticar peticiones HTTP mediante la cabecera `Authorization: Bearer`.
4. Diagnosticar y resolver errores comunes de seguridad como el bloqueo de solicitudes de pre-vuelo (CORS OPTIONS) y desajustes en el prefijo de autoridades de Spring Security.

Conceptos Clave (El 'Qué')

1. Manejo Centralizado de Excepciones y RFC 7807

En lugar de dispersar bloques `try-catch` en los controladores, la anotación `@RestControllerAdvice` intercepta de forma global cualquier excepción no controlada. Se utiliza el estándar **RFC 7807 (Problem Details)** para devolver objetos JSON homogéneos con campos como `title`, `status`, `detail`, `instance` y `timestamp`.

2. Autenticación y Autorización Basada en Roles (RBAC)

- **Autenticación:** Verifica quién es el usuario comparando credenciales con hashes `BCrypt` almacenados en la base de datos.

- **Autorización:** Evalúa los roles (ROLE_ADMIN, ROLE_OPERADOR) asignados al usuario para permitir o restringir el acceso a los endpoints.

3. Sesiones Sin Estado con JSON Web Tokens (JWT)

En APIs REST stateless (SessionCreationPolicy.STATELESS), el servidor no almacena sesiones en memoria. Al autenticarse, emite un token firmado criptográficamente (RFC 7519) compuesto de HEADER.PAYLOAD.SIGNATURE. El cliente envía este token en el encabezado `Authorization: Bearer <token>` de cada petición HTTP.

Parte 1: Práctica Guiada Paso a Paso

Paso 1: Configuración de Dependencias Maven (pom.xml)

Abra el archivo pom.xml de su proyecto Spring Boot y asegúrese de contar con las dependencias para Spring Security, Validación y JWT:

```
<dependencies>
  <!-- Spring Boot Starters -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <!-- Dependencias de JWT (JSON Web Token) -->
  <dependency>
```

```

        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.5</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.5</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.12.5</version>
        <scope>runtime</scope>
    </dependency>
    <!-- Driver de Microsoft SQL Server -->
    <dependency>
        <groupId>com.microsoft.sqlserver</groupId>
        <artifactId>mssql-jdbc</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>

```

Paso 2: Creación del Esquema Relacional de Seguridad en SQL Server

Abra SSMS y ejecute el siguiente script para crear las tablas de seguridad en su base de datos (reemplace [Carné] por su carnet universitario):

```

USE VideoRent[Carné]_II2026;

-- 1. Tabla de Usuarios
CREATE TABLE Usuario (
    usuario_id INT IDENTITY(1,1) PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    nombre_completo VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    activo BIT NOT NULL DEFAULT 1
);

```

```

-- 2. Tabla de Roles
CREATE TABLE Rol (
    rol_id INT IDENTITY(1,1) PRIMARY KEY,
    nombre_rol VARCHAR(30) NOT NULL UNIQUE
);

-- 3. Tabla Intermedia UsuarioRol
CREATE TABLE UsuarioRol (
    usuario_id INT NOT NULL,
    rol_id INT NOT NULL,
    PRIMARY KEY (usuario_id, rol_id),
    FOREIGN KEY (usuario_id) REFERENCES Usuario(usuario_id),
    FOREIGN KEY (rol_id) REFERENCES Rol(rol_id)
);

-- Inserción de Semillas Iniciales
-- Contraseña en texto plano para los usuarios de prueba: 'Password123!'
INSERT INTO Rol (nombre_rol)
VALUES ('ROLE_ADMIN'), ('ROLE_OPERADOR'), ('ROLE_CLIENTE');

INSERT INTO Usuario (username, password_hash, nombre_completo, email, activo)
VALUES ('admin',
        '$2a$10$e0MYzXyjpJS7Pd0RVvHwHe1Wn5cGBwA/7Xg0ymx1i86Kx5w5zK7y6',
        'Carlos Alvarado', 'admin@videorent.cr', 1);

INSERT INTO Usuario (username, password_hash, nombre_completo, email, activo)
VALUES ('operador',
        '$2a$10$e0MYzXyjpJS7Pd0RVvHwHe1Wn5cGBwA/7Xg0ymx1i86Kx5w5zK7y6',
        'María Rojas', 'operador@videorent.cr', 1);

-- Asignación de Roles
INSERT INTO UsuarioRol (usuario_id, rol_id) VALUES (1, 1); -- admin -> ROLE_ADMIN
INSERT INTO UsuarioRol (usuario_id, rol_id) VALUES (2, 2); -- operador -> ROLE_OPERADOR

```

Paso 3: Definición del DTO y Middleware Global de Excepciones

Cree la clase ErrorResponseDTO.java dentro del paquete dto:

```

package cr.ac.ucr.paraíso.ie.expresofast.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import java.time.LocalDateTime;

```

```

import java.util.Map;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponseDTO {
    private String title;
    private int status;
    private String detail;
    private String instance;
    private LocalDateTime timestamp;
    private Map<String, String> invalidFields;

    public ErrorResponseDTO(String title, int status, String detail,
                           String instance) {
        this.title = title;
        this.status = status;
        this.detail = detail;
        this.instance = instance;
        this.timestamp = LocalDateTime.now();
    }

    public ErrorResponseDTO(String title, int status, String detail,
                           String instance,
                           Map<String, String> invalidFields) {
        this(title, status, detail, instance);
        this.invalidFields = invalidFields;
    }

    public String getTitle() { return title; }
    public int getStatus() { return status; }
    public String getDetail() { return detail; }
    public String getInstance() { return instance; }
    public LocalDateTime getTimestamp() { return timestamp; }
    public Map<String, String> getInvalidFields() {
        return invalidFields;
    }
}

```

Cree la clase `GlobalExceptionHandler.java` anotada con `@RestControllerAdvice` dentro del paquete `exception`:

```

package cr.ac.ucr.paraíso.ie.expresofast.exception;

import cr.ac.ucr.paraíso.ie.expresofast.dto.ErrorResponseDTO;
import jakarta.servlet.http.HttpServletRequest;

```

```

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BadCredentialsException.class)
    public ResponseEntity<ErrorResponseDTO> handleBadCredentials(
        BadCredentialsException ex, HttpServletRequest request) {

        ErrorResponseDTO error = new ErrorResponseDTO(
            "Falla de Autenticación",
            HttpStatus.UNAUTHORIZED.value(),
            "Nombre de usuario o contraseña incorrectos.",
            request.getRequestURI()
        );
        return new ResponseEntity<>(error, HttpStatus.UNAUTHORIZED);
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponseDTO> handleAccessDenied(
        AccessDeniedException ex, HttpServletRequest request) {

        ErrorResponseDTO error = new ErrorResponseDTO(
            "Acceso Denegado",
            HttpStatus.FORBIDDEN.value(),
            "No posee los privilegios suficientes para este recurso.",
            request.getRequestURI()
        );
        return new ResponseEntity<>(error, HttpStatus.FORBIDDEN);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponseDTO> handleValidationErrors(
        MethodArgumentNotValidException ex, HttpServletRequest request) {

        Map<String, String> errors = new HashMap<>();
    }

```

```

        for (FieldError fieldError : ex.getBindingResult().getFieldErrors()) {
            errors.put(fieldError.getField(), fieldError.getDefaultMessage());
        }

        ErrorResponseDTO error = new ErrorResponseDTO(
            "Error de Validación",
            HttpStatus.BAD_REQUEST.value(),
            "Campos de la solicitud no cumplen las restricciones.",
            request.getRequestURI(),
            errors
        );
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponseDTO> handleGenericException(
        Exception ex, HttpServletRequest request) {

        ErrorResponseDTO error = new ErrorResponseDTO(
            "Error Interno del Servidor",
            HttpStatus.INTERNAL_SERVER_ERROR.value(),
            "Ocurrió un fallo inesperado. Contacte al administrador.",
            request.getRequestURI()
        );
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

Paso 4: Proveedor de Tokens JWT (`JwtTokenProvider.java`)

Cree la clase `JwtTokenProvider.java` en el paquete `security`:

```

package cr.ac.ucr.paraíso.ie.expresofast.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;

```

```

import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.stream.Collectors;

@Component
public class JwtTokenProvider {

    private final SecretKey key;
    private final long jwtExpirationInMs;

    public JwtTokenProvider(
        @Value("${app.jwt.secret:ClaveSecretaSuperSeguraParaJWTEmpresarial2026!}")
        String secret,
        @Value("${app.jwt.expiration-ms:86400000}")
        long jwtExpirationInMs) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.jwtExpirationInMs = jwtExpirationInMs;
    }

    public String generarToken(Authentication authentication) {
        String username = authentication.getName();
        Date ahora = new Date();
        Date expiracion = new Date(ahora.getTime() + jwtExpirationInMs);

        String roles = authentication.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .collect(Collectors.joining(","));

        return Jwts.builder()
            .subject(username)
            .claim("roles", roles)
            .issuedAt(ahora)
            .expiration(expiracion)
            .signWith(key)
            .compact();
    }

    public String obtenerUsernameDelJWT(String token) {
        Claims claims = Jwts.parser()
            .verifyWith(key)
            .build()
            .parseSignedClaims(token)
            .getPayload();
        return claims.getSubject();
    }
}

```

```

    public boolean validarToken(String token) {
        try {
            Jwts.parser().verifyWith(key).build().parseSignedClaims(token);
            return true;
        } catch (JwtException | IllegalArgumentException ex) {
            return false;
        }
    }
}

```

Paso 5: Filtro Interceptor de JWT (`JwtAuthenticationFilter.java`)

Cree la clase `JwtAuthenticationFilter.java` en el paquete `security`:

```

package cr.ac.ucr.paraíso.ie.expresofast.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtTokenProvider tokenProvider;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtTokenProvider tokenProvider,
                                   UserDetailsService userDetailsService) {
        this.tokenProvider = tokenProvider;
        this.userDetailsService = userDetailsService;
    }

    @Override

```

```

protected void doFilterInternal(HttpServletRequest request,
                                HttpServletResponse response,
                                FilterChain filterChain)
    throws ServletException, IOException {

    String token = obtenerJwtDeLaSolicitud(request);

    if (StringUtils.hasText(token) && tokenProvider.validarToken(token)) {
        String username = tokenProvider.obtenerUsernameDelJWT(token);

        UserDetails userDetails = userDetailsService.loadUserByUsername(username);
        UsernamePasswordAuthenticationToken auth =
            new UsernamePasswordAuthenticationToken(
                userDetails, null, userDetails.getAuthorities()
            );
        auth.setDetails(
            new WebAuthenticationDetailsSource().buildDetails(request)
        );

        SecurityContextHolder.getContext().setAuthentication(auth);
    }

    filterChain.doFilter(request, response);
}

private String obtenerJwtDeLaSolicitud(HttpServletRequest request) {
    String bearerToken = request.getHeader("Authorization");
    if (StringUtils.hasText(bearerToken) && bearerToken.startsWith("Bearer ")) {
        return bearerToken.substring(7);
    }
    return null;
}
}

```

Paso 6: Configuración de Seguridad Spring Security 6 (SecurityConfig.java)

Cree la clase de configuración SecurityConfig.java en el paquete config:

```

package cr.ac.ucr.paraíso.ie.expresofast.config;

import cr.ac.ucr.paraíso.ie.expresofast.security.JwtAuthenticationFilter;
import cr.ac.ucr.paraíso.ie.expresofast.security.JwtTokenProvider;

```

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    private final JwtTokenProvider tokenProvider;
    private final UserDetailsService userDetailsService;

    public SecurityConfig(JwtTokenProvider tokenProvider,
                        UserDetailsService userDetailsService) {
        this.tokenProvider = tokenProvider;
        this.userDetailsService = userDetailsService;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(10);
    }

    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
        http
            .cors(Customizer.withDefaults())

```

```

        .csrf(csrf -> csrf.disable())
        .sessionManagement(sess -> sess
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        )
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.OPTIONS, "**").permitAll()
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/peliculas/**")
                .hasAnyRole("ADMIN", "OPERADOR", "CLIENTE")
            .requestMatchers(HttpMethod.POST, "/api/peliculas/**")
                .hasAnyRole("ADMIN", "OPERADOR")
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .addFilterBefore(
            new JwtAuthenticationFilter(tokenProvider, userDetailsService),
            UsernamePasswordAuthenticationFilter.class
        );

    return http.build();
}
}

```

Paso 7: Controlador de Autenticación (AuthController.java)

Cree AuthController.java en el paquete controller:

```

package cr.ac.ucr.paraíso.ie.expresofast.controller;

import cr.ac.ucr.paraíso.ie.expresofast.dto.AuthRequestDTO;
import cr.ac.ucr.paraíso.ie.expresofast.dto.AuthResponseDTO;
import cr.ac.ucr.paraíso.ie.expresofast.security.JwtTokenProvider;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.web.bind.annotation.*;

import java.util.List;

```

```

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final JwtTokenProvider tokenProvider;

    public AuthController(AuthenticationManager authenticationManager,
                           JwtTokenProvider tokenProvider) {
        this.authenticationManager = authenticationManager;
        this.tokenProvider = tokenProvider;
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponseDTO> login(
        @Valid @RequestBody AuthRequestDTO loginDTO) {

        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                loginDTO.getUsername(), loginDTO.getPassword()
            )
        );

        String token = tokenProvider.generarToken(authentication);
        List<String> roles = authentication.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .toList();

        AuthResponseDTO response = new AuthResponseDTO(
            token, authentication.getName(), roles, 864000000L
        );
        return ResponseEntity.ok(response);
    }
}

```

Parte 2: Depuración y Diagnóstico de Errores Comunes

Diagnóstico 1: Bloqueo de Peticiones por CORS OPTIONS en Solicitudes Pre-flight

Al integrar el frontend desacoplado con el servidor protegido por JWT, el navegador emite automáticamente una solicitud pre-flight `OPTIONS`. Si Spring Security no permite la solicitud `OPTIONS` de forma explícita, se genera un rechazo HTTP 401/403.

- [] **Paso 1:** Abra las herramientas de desarrollador en su navegador (Pestaña *Network*).
- [] **Paso 2:** Ejecute una petición `POST` desde el cliente y verifique si la llamada pre-flight `OPTIONS` devuelve el código HTTP 401 `Unauthorized`.
- [] **Paso 3:** Confirme que en `SecurityConfig.java` se encuentre configurada la instrucción:

```
java .requestMatchers(HttpMethod.OPTIONS, "/*").permitAll()
```
- [] **Paso 4:** Reinicie el servidor Spring Boot y valide que la respuesta HTTP sea 200 `OK` con las cabeceras `Access-Control-Allow-Origin`.

Diagnóstico 2: Falla de Autorización por Desajuste en el Prefijo de Roles (ROLE_)

Un fallo común ocurre cuando la base de datos contiene la cadena "ADMIN" pero Spring Security invoca `hasRole("ADMIN")`, lo que deriva en una denegación de acceso 403 `Forbidden`.

- [] **Paso 1:** Verifique los logs del servidor al intentar consumir un endpoint protegido con `@PreAuthorize("hasRole('ADMIN')")`.
- [] **Paso 2:** Si observa una excepción `AccessDeniedException`, inspeccione las autoridades cargadas en `CustomUserDetailsService`.
- [] **Paso 3:** Asegúrese de que el nombre del rol recuperado de la base de datos incluya el prefijo `ROLE_` (ejemplo: `ROLE_ADMIN`) o agregue el prefijo dinámicamente al mapear los objetos `SimpleGrantedAuthority`:

```
java String nombreRol =  
rol.getNombreRol().startsWith("ROLE_") ? rol.getNombreRol() : "ROLE_" +  
rol.getNombreRol();
```

Parte 3: Reto Autónomo de Ampliación

Complete de forma autónoma el siguiente requerimiento técnico de ampliación sin guía paso a paso:

Requerimiento: Servicio de Lista Negra de Tokens y Logout Criptográfico

1. **Servicio `TokenBlacklistService.java`:**
2. Cree un servicio inyectable en memoria (`@Service`) respaldado por un conjunto concurrente (`ConcurrentHashMap.newKeySet()`) o una caché de Redis.

Implemente los métodos `public void revokeToken(String token)` y `public boolean estaRevocado(String token)`.

Endpoint de Cierre de Sesión (`POST /api/auth/logout`):

5. En `AuthController.java`, exponga el endpoint `POST /api/auth/logout` que requiera autenticación.

Extraiga el token del encabezado `Authorization: Bearer <token>` y regístrelo en el `TokenBlacklistService`.

Validación en `JwtAuthenticationFilter`:

8. Modifique el filtro para verificar que el token recibido **no** se encuentre registrado en la lista negra antes de autenticar al usuario en `SecurityContextHolder`.

Verificación de Éxito:

- Realice un inicio de sesión en `/api/auth/login` y obtenga el Token JWT.
- Utilice el token devuelto para consumir un endpoint protegido (`GET /api/peliculas`). Confirme la respuesta `200 OK`.
- Invoque el endpoint `POST /api/auth/logout` enviando el token JWT.
- Intente consumir nuevamente el endpoint protegido enviando el mismo token JWT. Verifique que el servidor rechace la petición devolviendo un código `401 Unauthorized` amigable.